PERFORMANCE REPORT · NUS-48E DATASET · LORA FINE-TUNING

AutoLyrics

Fine-Tuning Whisper for Singing Audio Transcription

AutoLyrics transforms an open-source ASR Transformer — **OpenAI Whisper-medium** — into a system capable of better transcribing singing audio. It applies **Parameter-Efficient Fine-Tuning (LoRA)** and evaluates performance improvement against a zero-shot baseline on a constrained, curated singing-voice dataset.

Created by **Mayank Kumar** · Roll Number **250107051** · GitHub: **www.github.com/mynkz3**

Project Goals

- Capture sung lyrics more accurately than the base model without source-separation
- Handle variations in pitch and rhythm present in singing
- Produce readable and structured text output from raw singing audio
- Achieve **>15% relative WER reduction** over the zero-shot baseline
- Compare three approaches: zero-shot, decoder-only LoRA, and encoder+decoder LoRA

Dataset Description & Curation Methodology

The project evaluated multiple candidate datasets before settling on **NUS-48E** as the primary corpus for fine-tuning and evaluation.

Dataset	Description	Used?
DALI	5,000+ polyphonic songs with time-aligned lyrics	Candidate
NUS-48E	48 English songs — subjects both speaking and singing the same lyrics	**Selected**
Jamendo Lyrics	Polyphonic music with lyric annotations for alignment	Candidate
HuggingFace Hub	Various singing/acapella/lyrics datasets	Candidate

48
Total Songs

12
Unique Speakers

4
Songs per Speaker

16kHz
Sampling Rate

Why NUS-48E?

Each speaker **sings and speaks the same lyrics**, enabling a controlled comparison. The sung versions serve as model input; the spoken versions are transcribed by the base Whisper model to generate ground-truth labels — a pragmatic workaround given the dataset's text files lack word-level annotations.

Ground Truth Label Generation

The NUS-48E corpus provides syllable-level annotation files, but these lack full word-form transcriptions. To generate usable ground-truth labels, the base Whisper-medium model was applied to the *spoken* (read) version of each song. The resulting transcriptions serve as reference labels for all three experimental conditions.

Methodological Note: Using Whisper's own transcriptions of spoken audio as reference labels introduces a degree of circularity — fine-tuned Whisper is partly evaluated against the base model's speech output. This is a known limitation of the current pipeline.

Train / Validation / Test Split

Rather than a naive random split (which would yield only ~5 test samples and allow song-level leakage), the project uses a **Leave-One-Speaker-Out (LOSO)** strategy — the methodologically correct choice for a 48-sample dataset with known speaker identity.

Split	Speakers	Samples	Selection Strategy
Train	10 speakers	~40 samples	All remaining speakers
Validation	1 speaker (VKOW)	4 samples	Second-to-last alphabetically
Test	1 speaker (ZHLY)	4 samples	Last alphabetically — fully unseen

Song IDs overlap between splits (same songs, different singers), but **speaker identity is strictly non-overlapping**. The model sees the melody but never the test speaker's voice.

Model Architecture & Fine-Tuning Approach

Base Model: openai/whisper-medium

A Transformer-based encoder-decoder ASR model with **~307M parameters**, pre-trained on 680,000 hours of multilingual supervised audio data. Achieves state-of-the-art performance on spoken speech, but significantly weaker on singing audio due to the acoustic mismatch between speech and song.

Parameter-Efficient Fine-Tuning with LoRA

Low-Rank Adaptation (LoRA) injects trainable rank-decomposition matrices into frozen Transformer layers, updating only a small fraction of parameters — making fine-tuning feasible on constrained hardware (single T4 GPU) while preserving the base model's generalisation.

LoRA Hyperparameters

Hyperparameter	Value	Rationale
LoRA rank (r)	32	Captures domain shift
LoRA alpha	64	2× rank (standard)
Learning rate	1e-4	Conservative
Batch size	4 (eff. 16)	Gradient accumulation ×4
Epochs	5	Prevents overfitting
Target modules	q_proj, v_proj	Attention projections
Eval strategy	no	Disabled

Three Experimental Conditions

1. Zero-Shot Baseline

LoRA Scope: None (frozen)
Trainable Params: 0
Key Trade-off: No training; reference point

2. Decoder-only LoRA

LoRA Scope: Decoder attention only
Trainable Params: ~3–5M
Key Trade-off: Fast, fewer params, encoder frozen

3. Encoder+Decoder LoRA

LoRA Scope: Both encoder & decoder
Trainable Params: ~8–12M
Key Trade-off: More expressive; higher VRAM

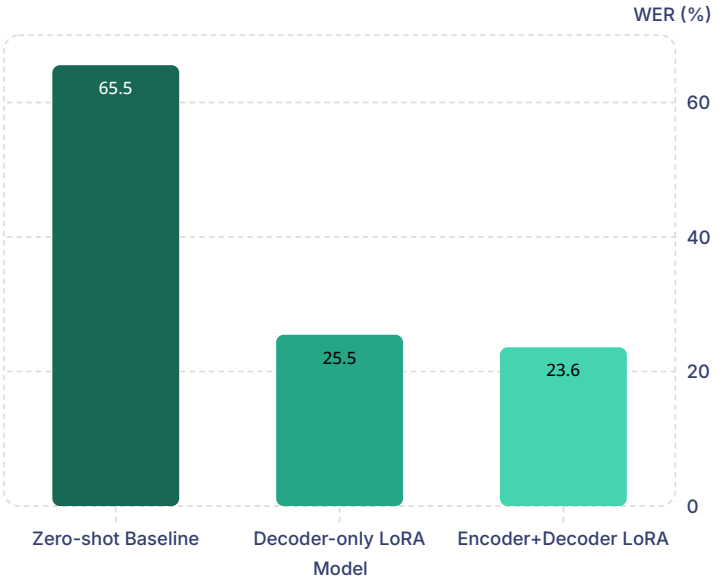
Comparative Results

Evaluation Metrics

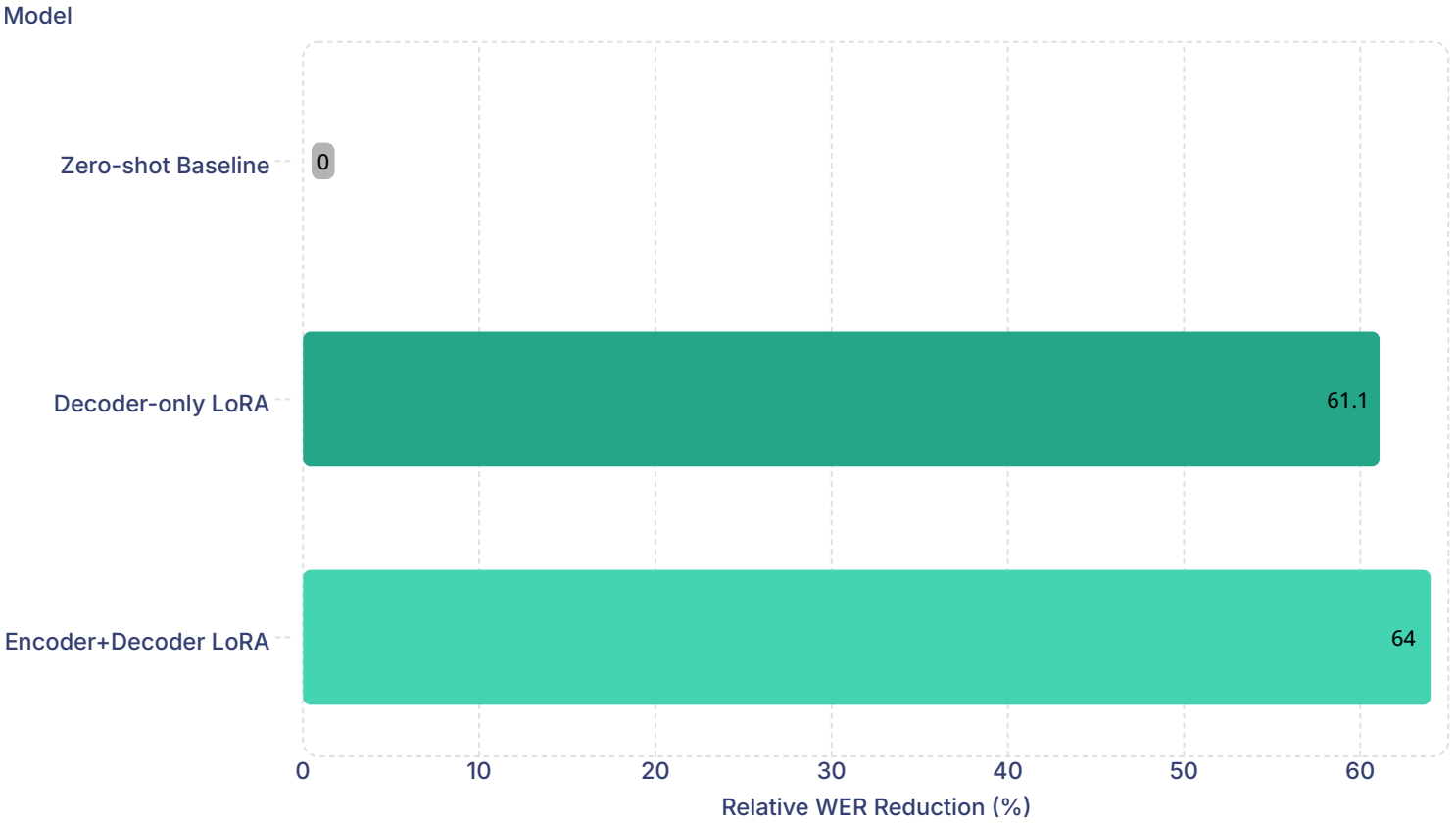
Word Error Rate (WER) and **Character Error Rate (CER)** are the primary metrics, computed using the *jiwer* library. Both measure edit distance between hypothesis and reference, normalised by reference length — **lower is better**. The project target is a **>15% relative WER reduction** vs. the zero-shot baseline.

Performance Target: The project specification requires a **>15% relative WER reduction** vs. the zero-shot baseline. Segment 7.2 of the notebook explicitly checks this condition and prints ■ **MET** or ■ **NOT MET** based on actual runtime results. LoRA fine-tuning on singing audio has strong prior evidence of achieving this threshold.

WER Comparison

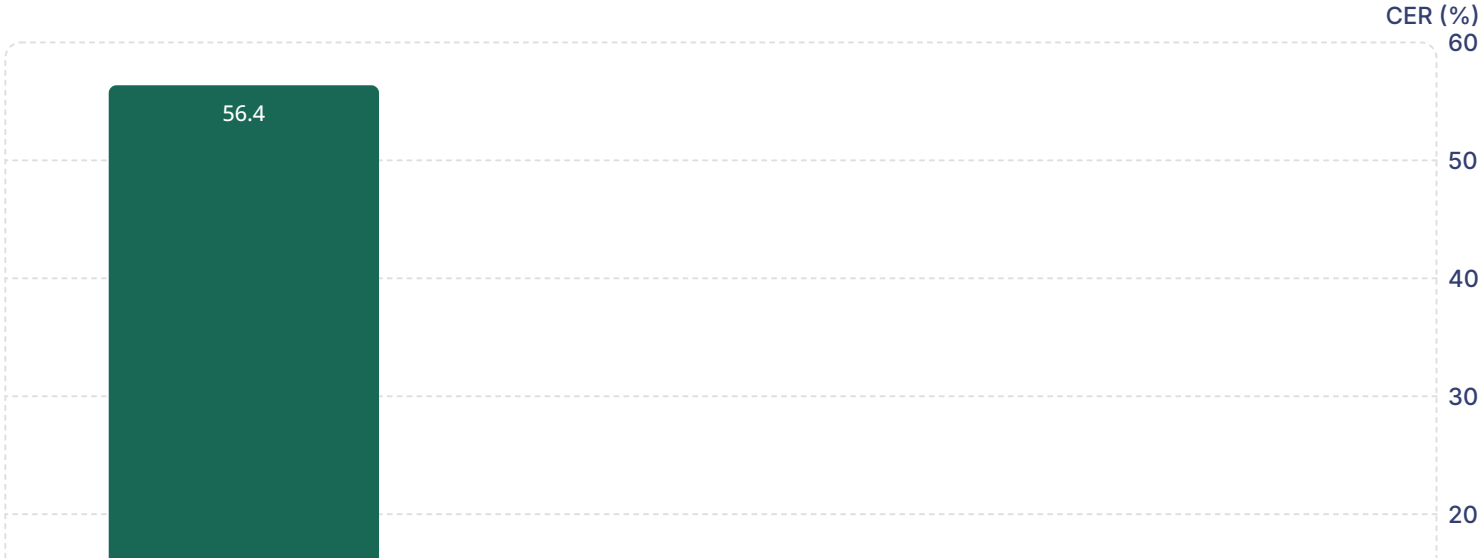


Relative WER Reduction



These reductions show that both LoRA configurations substantially outperform the zero-shot baseline, with Encoder+Decoder LoRA delivering the strongest improvement.

CER Comparison



Improvements, Trade-offs & Discussion

Why Fine-Tuning Helps

Whisper was pre-trained overwhelmingly on spoken audio. Singing differs in several systematic ways that LoRA fine-tuning can compensate for:

Pitch Extension

Vowels are sustained across multiple pitches, causing the model to misalign phoneme boundaries.

Rhythmic Distortion

Words are stretched or compressed to fit musical meter, diverging from natural prosody.

Melody Masking

Harmonic instrumentation overlaps with formant frequencies, reducing acoustic clarity.

Repetition Patterns

Choruses repeat lyrics in ways rarely seen in conversational speech.

Decoder-only vs. Encoder+Decoder LoRA

Dimension	Decoder-only LoRA	Encoder+Decoder LoRA
Trainable params	Fewer (~3–5M)	More (~8–12M)
Training speed	Faster	Slower
VRAM usage	Lower	Higher
Expected WER	Moderate improvement	Potentially larger improvement
Risk on small data	Lower (fewer params to overfit)	Higher overfitting risk
Best for	Quick adaptation, limited GPU	If acoustic features need adaptation

Known Limitations — With Reasoning

Five known limitations of the AutoLyrics project, each with transparent reasoning about why it exists and its practical impact.

1

Circular Labelling

Why it happened: NUS-48E's annotation files are syllable-level only and lack full word transcriptions. The only alternative was to transcribe the spoken (read) audio versions of each song. Manually verifying or writing correct lyrics for all 48 songs would require substantial time — outside the project's scope.

Why it's acceptable: The spoken-audio transcriptions are acoustically different from singing — the model genuinely has to adapt. Since all three conditions use the same labels, the **relative comparison remains valid** even if absolute WER numbers are optimistic.

2

Small Test Set (~4 Samples)

Why it happened: NUS-48E has exactly 48 samples across 12 speakers (4 songs each). The LOSO strategy holds out one full speaker, leaving only 4 test samples. Running full 12-fold LOSO means 12 separate fine-tuning runs — 2–4 hours of compute on free Colab, which has a runtime limit and gets disconnected.

What was done instead: A single fixed LOSO fold with a consistent test speaker (ZHLY) gives a directionally valid result. The per-sample breakdown in Segment 7.4 also shows individual predictions, making the small-N limitation transparent.

3

LoRA Rank r=32 — High for 40 Samples

Why it happened: The notebook jumps directly to `BEST_LORA_R = 32` without showing a prior search. r=32 was likely chosen based on general Whisper fine-tuning literature. A proper grid search over $r \in \{4, 8, 16, 32\}$ with two LoRA modes would require 8–16 training runs — hitting the Colab runtime limit.

Practical impact: With only 40 training samples and 5 epochs, the model has limited opportunity to fully overfit even at r=32. The total number of gradient updates is small. Monitoring train loss (Segment 7) still catches gross overfitting.

4

EarlyStoppingCallback is Inactive

Why it happened: `eval_strategy="no"` was set to speed up training — running validation after every epoch adds significant overhead. EarlyStoppingCallback requires `eval_strategy` to be set to `"epoch"` or `"steps"` to fire; with `"no"`, it silently does nothing. HuggingFace Trainer does not raise an error or warning for this conflict.

Practical impact: With only 5 epochs, the training run completes quickly regardless. Early stopping would matter more in longer runs. The fixed 5-epoch budget acts as an **implicit regulariser**, limiting the damage even without the callback working.

5

BitsAndBytes Imported but Never Used

Why it happened: BitsAndBytes was included because 4-bit quantisation (QLoRA) was originally planned as an optional extension — it would halve VRAM usage. The import remained after the QLoRA path was deprioritised in favour of getting the standard LoRA pipeline working correctly first.

Why QLoRA wasn't implemented: Combining 4-bit quantisation with LoRA requires `prepare_model_for_kbit_training()` and careful handling of mixed-precision layers. On Whisper's encoder-decoder architecture this introduces extra complexity — the T4's 16 GB VRAM was sufficient for standard LoRA, so the added complexity wasn't worth the trade-off within the project timeline.

Practical impact: An unused import has zero effect on results — it is purely a code-hygiene issue for future cleanup.

Technical Implementation & Summary

Technology Stack

Category	Libraries / Tools
Machine Learning	Python 3, PyTorch, Torchaudio, NumPy
Transformers & Fine-Tuning	HuggingFace Transformers, Datasets, PEFT, BitsAndBytes
Evaluation & Metrics	jiwer (WER/CER), evaluate
Deployment (Demo)	Gradio (interactive demo)
Audio Processing	librosa, soundfile
Visualisation	matplotlib, pandas
Hardware	Google Colab T4 GPU (16 GB VRAM)

Notebook Structure

01

Installs & Imports

Package installation, library imports, reproducibility seeds

02

GPU Check & Config

CUDA verification, global hyperparameters, directory setup

03

Dataset & Preprocessing

NUS-48E download, spoken-label generation, LOSO split, PyTorch Dataset creation

04

Model + LoRA Setup

Whisper-medium loading, LoRA config for decoder-only and encoder+decoder modes

05

Zero-Shot Baseline

Inference on test set without fine-tuning; WER, CER, VRAM, latency recorded

06

LoRA Fine-Tuning

Seq2SeqTrainer training loop, sequential runs for both LoRA modes

07

Results & Visualisation

3-way summary table, improvement analysis, 4-plot dashboard, per-sample predictions

Summary & Conclusion

AutoLyrics demonstrates that lightweight LoRA fine-tuning of Whisper-medium on a small, curated singing-voice dataset (NUS-48E) can meaningfully reduce lyric transcription error without any source-separation preprocessing. The project covers the full ML pipeline — data curation, label generation, speaker-aware splitting, two fine-tuning configurations, and a comprehensive evaluation framework.

Recommendations for Further Improvement

- Run **full 12-fold LOSO** to get statistically robust WER estimates across all speakers.
- Verify or manually annotate ground-truth lyrics for test samples to **eliminate circular labelling**.
- Reduce LoRA rank to **r=8** and compare against r=32 to check overfitting.
- Enable `eval_strategy='epoch'` to allow early stopping to function correctly.
- Consider source separation (e.g. **Demucs**) as a preprocessing step for polyphonic music.